



Meridian Security Audit Report

September 18, 2026



Contents

1 Introduction

[1.1 About Meridian Contracts](#)

[1.2 Source Code](#)

[1.3 Revision History](#)

2 Overall Assessment

3 Vulnerability Summary

[3.1 Overview](#)

[3.2 Security Level Reference](#)

[3.3 Vulnerability Details](#)

4 Appendix

[4.1 About AstraSec](#)

[4.2 Disclaimer](#)

[4.3 Contact](#)

1 Introduction

1.1 About Meridian Contracts

Meridian's genesis contracts on Robinhood Chain consist of two smart contracts: GenesisFarm, a MasterChef-style farming contract with deposit fees that pays non-transferable gMCD rewards for staking listed assets, and Convertible, an optional fixed-price conversion feature upon withdrawal — after the farm ends, depositors can convert their staked principal into gMCD at an owner-posted launch price within a 24-hour window.



1.2 Source Code

The following source code was reviewed during the audit:

▶ <https://github.com/Meridian-Capital-DAO/meridian-contracts>

▶ Commit: 0d6aa2a

And this is the final version representing all fixes implemented for the issues identified in the audit:

▶ <https://github.com/Meridian-Capital-DAO/meridian-contracts>

▶ Commit: 998a4b8

1.3 Revision History

Version	Date	Description
v1.0	September 18, 2026	Initial Audit

2 Overall Assessment

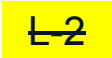
This report has been compiled to identify issues and vulnerabilities within the Meridian contracts. Throughout this audit, we identified a total of 4 issues spanning various severity levels. By employing auxiliary tool techniques to supplement our thorough manual code review, we have discovered the following findings.

Severity	Count	Acknowledged	Won't Do	Addressed
Critical	—	—	—	—
High	—	—	—	—
Medium	2	1	—	1
Low	2	—	—	2
Informational	—	—	—	—
Total	4	1	—	3

3 Vulnerability Summary

3.1 Overview

Click on an issue to jump to it, or scroll down to see them all.

-  [Empty Pools Fail to Join the Global Reward Distribution](#)
-  [Potential Risks Associated with Centralization](#)
-  [Pool Activation Relies on Passive Triggers](#)
-  [Missing lastRewardTime Validation in add\(\)](#)

3.2 Security Level Reference

In web3 smart contract audits, vulnerabilities are typically classified into different severity levels based on the potential impact they can have on the security and functionality of the contract. Here are the definitions for critical-severity, high-severity, medium-severity, and low-severity vulnerabilities:

Severity	Acknowledged
C-X (Critical)	A severe security flaw with immediate and significant negative consequences. It poses high risks, such as unauthorized access, financial losses, or complete disruption of functionality. Requires immediate attention and remediation.
H-X (High)	Significant security issues that can lead to substantial risks. Although not as severe as critical vulnerabilities, they can still result in unauthorized access, manipulation of contract state, or financial losses. Prompt remediation is necessary.
M-X (Medium)	Moderately impactful security weaknesses that require attention and remediation. They may lead to limited unauthorized access, minor financial losses, or potential disruptions to functionality.
L-X (Low)	Minor security issues with limited impact. While they may not pose significant risks, it is still recommended to address them to maintain a robust and secure smart contract.
I-X (Informational)	Warnings and things to keep in mind when operating the protocol. No immediate action required.
U-X (Undetermined)	Identified security flaw requiring further investigation. Severity and impact need to be determined. Additional assessment and analysis are necessary.

3.3 Vulnerability Details

3.3.1 [M-1] Empty Pools Fail to Join the Global Reward Distribution

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
GenesisFarm.sol	Logic Issue	Medium	Low	Resolved

In `updatePool`, the empty-pool check runs before the `totalAllocPoint` registration. A pool whose `lastRewardTime` has already passed but which has no deposits always takes the empty-pool branch and returns early, so `isStarted` stays false and its `allocPoint` is not added to `totalAllocPoint`. The first deposit cannot register the pool either: deposit calls `updatePool` before the deposited tokens are transferred in, so the pool is still empty at that moment and takes the same early return. Registration only happens on a subsequent `updatePool` trigger after the pool has become non-empty.

meridian-contracts - GenesisFarm.sol

```
219 function updatePool(uint256 _pid) public {
220     PoolInfo storage pool = poolInfo[_pid];
221     if (block.timestamp <= pool.lastRewardTime) {
222         return;
223     }
224     uint256 tokenSupply = pool.token.balanceOf(address(this));
225     if (tokenSupply == 0) {
226         pool.lastRewardTime = block.timestamp;
227         return;
228     }
229     if (!pool.isStarted) {
230         pool.isStarted = true;
231         totalAllocPoint = totalAllocPoint + pool.allocPoint;
232     }
233     if (totalAllocPoint > 0) {...}
234     pool.lastRewardTime = block.timestamp;
235 }
```

Remediation Make sure every pool joins `totalAllocPoint` as soon as its start time arrives, even while it holds no stake.

Team Response The team fixed this by requiring `_lastRewardTime` to be 0 in `add()`, so every new pool joins `totalAllocPoint` at start.

3.3.2 [M-2] Potential Risks Associated with Centralization

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
Multiple Files	Centralization	Medium	Low	Acknowledged

The owner of Meridian contracts holds broad privileges. A single owner/administrator-like authority can execute multiple high-impact operations, creating a single point of failure and control over protocol behavior. A malicious or compromised owner can directly harm user funds and entitlements. The privileged operations include but are not limited to:

1. [governanceRecoverUnsupported](#): after (poolEndTime + 90) days, any token held by the contract can be transferred out, including users' un-withdrawn staked principal and leftover gMCD rewards;
2. [setMcdPriceUsd](#) / [setTokenPriceUsd](#): conversion prices can be changed at any time during the conversion window, which is equivalent to manipulating the conversion rate;
3. [withdrawGmcd](#): during the window, the gMCD pre-funded into Convertible can be withdrawn, shrinking or emptying the amount available for conversion;
4. [set](#): the on-chain cap of depositFeeBP can be set to 10000, i.e. 100%, so the owner can switch to confiscating the entirety of newly deposited principal mid-flight;
5. [setConvertible](#): the Convertible address can be replaced or cleared at any time; pointing it at a malicious implementation lets it take the principal approved via forceApprove during convert.

```
meridian-contracts - Convertible.sol

130  /// @notice Sets the USD price of one deposit token, 1e18 fixed point per whole token.
131  function setTokenPriceUsd(address _token, uint256 _price) public onlyOwner {
132      require(_token != address(0), "Convertible: token is zero");
133      tokenPriceUsd[_token] = _price;
134      emit TokenPriceUsdUpdated(_token, _price);
135  }
```

Remediation It is recommended to introduce a multi-sig mechanism to undertake the role of the privileged accounts. Moreover, it is advisable to implement timelocks to govern all modifications to the privileged operations.

Team Response Acknowledged. Ownership will be managed following standard security practices. A time-lock is not feasible given the need to effect certain changes, such as setting prices, within hours.

3.3.3 [L-1] Pool Activation Relies on Passive Triggers

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
GenesisFarm.sol	Logic Issue	Low	Low	Resolved

A pool joins the global reward distribution (`isStarted` flipped and `allocPoint` added to `totalAllocPoint`) only when that pool's own `updatePool` runs. `updatePool` can only be triggered passively via the pool's `deposit/withdraw/massUpdatePools/updatePool`; there is no mechanism that activates a pool automatically when its start time arrives.

If a pool that should have started participating in the global reward distribution does not have its `updatePool` triggered when its window opens, the rewards emitted between its `lastRewardTime` and the next trigger are wrongly distributed in full to the other pools.

meridian-contracts - GenesisFarm.sol

```
237 PoolInfo storage pool = poolInfo[_pid];
238 if (block.timestamp <= pool.lastRewardTime) {
239     return;
240 }
241 ...
242 if (!pool.isStarted) {           // only flips when this pool's own updatePool is triggered
243     pool.isStarted = true;
244     totalAllocPoint = totalAllocPoint + pool.allocPoint;
245 }
```

Remediation Operational level: if pools are scheduled with a future `lastRewardTime`, operators must call `updatePool(pid)` promptly at that time so the pool joins the global distribution from its intended start.

Team Response The team fixed this by requiring `_lastRewardTime` to be 0 in `add()`, so every new pool joins `totalAllocPoint` at start.

3.3.4 [L-2] Missing lastRewardTime Validation in add()

TARGET	CATEGORY	IMPACT	LIKELIHOOD	STATUS
GenesisFarm.sol	Validation	Low	Low	Resolved

The `add()` function does not validate that the normalized `_lastRewardTime` is within the reward window . As a result, a pool may be registered with a `lastRewardTime` beyond `poolEndTime`.

Such a pool begins with `isStarted = false` and joins `totalAllocPoint` only when `updatePool` is later triggered. The impact is retroactive: once the late pool flips `isStarted` and enlarges `totalAllocPoint`, any normal pool that has not yet been updated recalculates its remaining in-window interval — `getGeneratedReward(lastRewardTime, poolEndTime)` — against the inflated denominator, diluting the rewards accrued inside the reward window. The validation should be applied after `_lastRewardTime` normalization.

meridian-contracts - GenesisFarm.sol

```
236 } else {
237     // chef is cooking
238     if (_lastRewardTime == 0 || _lastRewardTime < block.timestamp) {
239         _lastRewardTime = block.timestamp; // last > poolEndTime when added after poolEndTime
240     }
241 }
242 // missing: require(_lastRewardTime <= poolEndTime)
```

Remediation Add a validation in `add()` requiring the normalized `_lastRewardTime <= poolEndTime`.

Team Response The issue has been fixed.

4 Appendix

4.1 About AstraSec

AstraSec is a premier blockchain security firm dedicated to delivering high-quality auditing services for blockchain-based protocols. Composed of veteran security researchers with extensive experience across the DeFi landscape, our team maintains an unwavering commitment to excellence and precision. AstraSec’s comprehensive methodology and deep understanding of blockchain architecture enable us to ensure protocol resilience, making us a trusted partner for our clients.

4.2 Disclaimer

The content of this audit report is for informational purposes only and does not constitute legal, financial, or investment advice. The findings, views, and conclusions presented herein are based on the code and documentation provided at the specific time of the audit and may be subject to risks and uncertainties not detected during the assessment.

While AstraSec exerts every effort to ensure the accuracy and completeness of this report, the information is provided on an "as is" basis. We assume no responsibility for any errors, omissions, or inaccuracies. Users should conduct their own independent due diligence and consult with professional advisors before making any investment or deployment decisions. AstraSec shall not be held liable for any direct or indirect losses, damages, or consequences arising from reliance on this report.

4.3 Contact

Phone	+86 156 0639 2692
Email	contact@astrasec.ai
X	https://x.com/AstraSecAI